

COMPILER DESIGN

PROGRAMMING EXERCISE 5 :

MAANYA BHARDWAJ
PES2UG23CS311

LEXER.L

```
%{
#define YYSTYPE char*
#include <unistd.h>
#include "parser.tab.h"
#include <stdio.h>
extern void yyerror(const char *); // declare the error handling function
%}
/* Regular definitions */
digit [0-9]
letter [a-zA-Z]
id {letter}({letter}|{digit})*
digits {digit}+
opFraction (\.{digits})?
opExponent ([Ee][+-]?{digits})?
number {digits}{opFraction}{opExponent}
%option yylineno
%%
\\(.*) ; // ignore comments
[\\t\\n] ; // ignore whitespaces
"(" {return *yytext;}
")" {return *yytext;}
"." {return *yytext;}
"," {return *yytext;}
```

```

"*" {return *yytext;}
"+" {return *yytext;}
";" {return *yytext;}
"-" {return *yytext;}
"/" {return *yytext;}
"=" {return *yytext;}
">" {return *yytext;}
"<" {return *yytext;}
{number} {
    yylval = strdup(yytext); //stores the value of the number to be used
    later for symbol table insertion

```

```

return T_NUM;
}
{id} {
    yylval = strdup(yytext); //stores the identifier to be used later for
    symbol table insertion

```

```

return T_ID;
}
. {} // anything else => ignore
%%
int yywrap()
{
    return 1;
}

```

PARSER.Y

```

%{
#include "abstract_syntax_tree.h"

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void yyerror(char* s);
// error handling function
int yylex();
// declare the function performing lexical analysis
extern int yylineno;
// track the line number
%}
%union
// union to allow nodes to store return different datatypes
{
char* text;
expression_node* exp_node;
}
%token <text> T_ID T_NUM
%type <exp_node> E T F
/* specify start symbol */
%start START
%%
START : ASSGN {
printf("Valid syntax\n");
YYACCEPT;
// If program fits the grammar, syntax is valid
}
/* Grammar for assignment */
ASSGN : T_ID '=' E {
// displaying the expression tree
printf("%s\n", $1);
display_exp_tree($3);
printf("\n");
}

```

```

}
;
/* Expression Grammar */
E : E '+' T {

    $$ = init_exp_node("+", $1, $3);
}
| E '-' T {

    $$ = init_exp_node("-", $1, $3);
}
| T { $$ = $1; }
;
T : T '*' F {

    $$ = init_exp_node("*", $1, $3);
}
| T '/' F {

    $$ = init_exp_node("/", $1, $3);
}
| F {
    //pass AST node to the parent
    $$ = $1;
}
;
F : '(' E ')' { $$ = $2; }
| T_ID {
    // creating a terminal node of the AST
    $$ = init_exp_node($1, NULL, NULL);
}
| T_NUM {

```

```

// creating a terminal node of the AST
$$ = init_exp_node($1, NULL, NULL);
}
;
%%
/* error handling function */
void yyerror(char* s)
{
printf("Error :%s at %d \n",s,yylineno);
}
/* main function - calls the yyparse() function which will in turn drive
yylex() as well */
int main(int argc, char* argv[])
{
yyparse();
return 0;
}

```

ABSTRACT_SYNTAX_TREE.C

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "abstract_syntax_tree.h"

expression_node* init_exp_node(char* val, expression_node* left, expression_node*
right)
{
// function to allocate memory for an AST node and set the left and right children of
the nodes
expression_node* node = (expression_node*)malloc(sizeof(expression_node));
node->val = strdup(val);
node->left = left;
node->right = right;
return node;
}

void display_exp_tree(expression_node* exp_node)

```

```

{
// traversing the AST in preorder and displaying the nodes
if (exp_node == NULL) return;
printf("%s\n", exp_node->val);
fflush(stdout);
display_exp_tree(exp_node->left);
display_exp_tree(exp_node->right);
}

```

ABSTRACT_SYNTAX_TREE.H

```

typedef struct expression_node
{
/* needs 3 members
i) pointer to the left child
ii) pointer to the right child
iii) string to store the value of the node
*/
struct expression_node* left;
struct expression_node* right;
char* val;
}expression_node;
expression_node* init_exp_node(char* val, expression_node* left, expression_node*
right);
void display_exp_tree(expression_node* exp_node);

```

TEST_INPUT_1.C

```

a = 10 / 5 - 2 * 7 + 3

```

TEST_INPUT_2.C

```

b = c / 6.7 + 12.45 - a * 1234.0

```

OUTPUT

```
maanyabhardwaj@Maanyabhardwaj PE5 % /opt/homebrew/opt/flex/bin/flex lexer.l
/opt/homebrew/opt/bison/bin/bison -d parser.y
gcc parser.tab.c lex.yy.c abstract_syntax_tree.c -o parser
./parser < test_input_1.c
a
+
-
/
10
5
*
2
7
3

Valid syntax
maanyabhardwaj@Maanyabhardwaj PE5 % /opt/homebrew/opt/flex/bin/flex lexer.l
/opt/homebrew/opt/bison/bin/bison -d parser.y
gcc parser.tab.c lex.yy.c abstract_syntax_tree.c -o parser
./parser < test_input_2.c
b
-
+
/
c
6.7
12.45
*
a
1234.0

Valid syntax
maanyabhardwaj@Maanyabhardwaj PE5 %
```